

Сравнение и анализ оптимизаций компиляторов

Тарадеев Андрей Михайлович

Группа: 25.Б41-мм

Кафедра: кафедра системного программирования

Научный руководитель: Инженер-исследователь Романова Зинаида Андреевна

Номер семестра практики: 2

1. Постановка задачи

Целью работы является исследование и сравнительный анализ стратегий оптимизации машинного кода, применяемых современными компиляторами языка Си, на примере программы `optbench.c`.

Для достижения цели поставлены следующие задачи:

1. описать среду исследования (ОС, архитектура, версии компиляторов);
2. выполнить компиляцию тестовой программы с уровнями оптимизации `-O0`, `-O2`, `-Os`;
3. произвести замеры размера исполняемых файлов и времени выполнения;
4. сгенерировать ассемблерные листинги и составить сравнительные таблицы;
5. провести анализ применённых оптимизаций по каждому компилятору;
6. сравнить компиляторы между собой и сформулировать выводы.

В качестве объектов исследования выбраны три широко распространённых компилятора: GCC 13.3.0 (GNU Compiler Collection), Clang 18.1.3 и Intel ICX 2026.0.0. Все три установлены на платформе Ubuntu 24.04 LTS (x86_64). Характеристики системы и точные версии инструментов зафиксированы в репозитории^{1,2}.

Тестовой программой послужил бенчмарк PC Tech Journal 1988 года³ (`optbench.c`), разработанный для проверки компиляторных оптимизаций. Программа охватывает девять классических сценариев: свёртку констант, арифметические тождества, снижение мощности операций, удаление мёртвого кода, управление переменной индукции цикла, вынесение инвариантов из цикла, устранение общих подвыражений, размотку цикла и сжатие цепочек переходов.

Результаты работы полезны при выборе компилятора и параметров оптимизации в зависимости от приоритетов проекта: скорость выполнения, размер кода или отлаживаемость.

2. Описание предлагаемого решения

Методика исследования включала четыре последовательных этапа.

Этап 1 — сборка исполняемых файлов. Для каждого из трёх компиляторов программа собиралась с тремя уровнями оптимизации: `-O0` (отладочная сборка, без оптимизации), `-O2` (агрессивная оптимизация по скорости) и `-Os` (оптимизация по объёму кода). Во всех случаях добавлялся флаг `-DNO_ZERO_DIVIDE`, исключающий блок с делением на ноль, который препятствует компиляции. Итого получено 9 исполняемых файлов. Для каждого фиксировались: размер командой `ls -la` и суммарное время десяти запусков:

```
time for i in $(seq 1 10); do ./binary > /dev/null; done
```

Этап 2 — генерация ассемблерных листингов. Флаг `-S` применялся для получения текстового ассемблерного представления. Листинги создавались для уровней `-O0` и `-O2`; итого шесть файлов. Команды сборки:

```
gcc -O0 -S -DNO_ZERO_DIVIDE src/optbench.c -o asm/gcc/gcc_00.s
gcc -O2 -S -DNO_ZERO_DIVIDE src/optbench.c -o asm/gcc/gcc_02.s
clang -O0 -S -DNO_ZERO_DIVIDE src/optbench.c -o asm/clang/clang_00.s
clang -O2 -S -DNO_ZERO_DIVIDE src/optbench.c -o asm/clang/clang_02.s
icx -O0 -S -DNO_ZERO_DIVIDE src/optbench.c -o asm/icx/icx_00.s
icx -O2 -S -DNO_ZERO_DIVIDE src/optbench.c -o asm/icx/icx_02.s
```

Все листинги сохранены в репозитории⁴.

Этап 3 — покодový анализ листингов. Каждый листинг изучался применительно к 14 типам оптимизаций, явно выраженных в исходном коде. Для каждого компилятора составлена таблица с тремя колонками: исходная конструкция на Си, ассемблер при `-O0` и ассемблер при `-O2`. К каждой строке дано пояснение: что именно изменилось, почему это сделано компилятором и насколько замена логична.

Этап 4 — сравнение компиляторов. На основе трёх таблиц составлена сводная матрица оптимизаций: по строкам — типы преобразований, по столбцам — компиляторы. В ней отражено, какие оптимизации поддерживает каждый компилятор, где они уникальны и где идентичны.

3. Эксперименты

3.1. Размер бинарных файлов

Полные таблицы размеров для каждого компилятора приведены в репозитории⁵.

¹ Характеристики системы: https://github.com/Andrew2114/Compiler_optimization/blob/main/tables/01_system_info.md.

² Версии компиляторов: https://github.com/Andrew2114/Compiler_optimization/blob/main/tables/02_compiler_versions.md.

³ Исходный код `optbench.c`: https://github.com/Andrew2114/Compiler_optimization/blob/main/src/optbench.c.

⁴ Ассемблерные листинги GCC, Clang, ICX: https://github.com/Andrew2114/Compiler_optimization/tree/main/asm.

⁵ Таблицы размеров файлов — GCC, Clang, ICX: https://github.com/Andrew2114/Compiler_optimization/blob/main/tables/03_1_executable_files.md, https://github.com/Andrew2114/Compiler_optimization/blob/main/tables/03_2_executable_files.md, https://github.com/Andrew2114/Compiler_optimization/blob/main/tables/03_3_executable_files.md.

GCC сформировал файлы одинакового размера (16 992 байт) при всех трёх флагах. Это объясняется тем, что для данной небольшой программы большую часть объёма ELF-файла занимают заголовки, таблицы символов и секции метаданных, а не машинный код: оптимизация последнего не меняет итоговый размер ощутимо.

Clang при `-O0` совпал с GCC (16 992 байт), однако при `-O2` и `-Os` вырос до 17 048 байт. Прирост в 56 байт объясняется добавлением секции `.address` (Address Significance Table) — таблицы, которую использует компоновщик LLD для оптимизации ссылок на глобальные символы. GCC и ICX эту секцию не генерируют.

ICX при `-O0` дал наименьший размер (16 936 байт): компилятор не добавляет расширенных отладочных метаданных по умолчанию. При `-O2` размер незначительно вырос до 16 984 байт из-за инлайнинга вспомогательных функций в `main` и добавления пролога инициализации FPU.

Вывод. Для данного бенчмарка ни у одного компилятора флаги оптимизации не дали ощутимого сокращения размера — программа слишком мала. Наименьший размер обеспечил ICX (`-O0`), наибольший — Clang (`-O2/-Os`).

3.2. Время выполнения

Подробные данные по всем девяти вариантам сборки приведены в репозитории⁶. Время одного запуска вычислялось как суммарное `real` делённое на 10. Столбец `user` отражает время непосредственного выполнения программы, `sys` — время на системные вызовы (`printf`, запуск процесса).

GCC показал наибольшее ускорение: с $\approx 3,9$ мс (`-O0`) до $\approx 3,4$ мс (`-O2`) — ≈ 13 %. Флаги `-O2` и `-Os` дали одинаковый результат. У Clang разница между `-O0` (3,6 мс) и `-O2` (3,5 мс) минимальна — менее 3 %; `-Os` не дал ускорения относительно `-O0`. ICX при `-O0` оказался самым медленным (4,0 мс) — из-за инструкций инициализации SSE-блока (`stmxcsr/ldmxcsr`) в `main`; при `-O2` — 3,9 мс, при `-Os` — 3,7 мс.

Вывод. Различия между флагами невелики: программа многократно вызывает `printf`, и до 90 % времени составляют системные вызовы (`sys`), а не вычислительный код. Для корректного измерения вычислительных оптимизаций следует использовать программу без интенсивного ввода-вывода.

3.3. Анализ ассемблерных листингов

Три таблицы с поковым анализом (по одной на компилятор) размещены в репозитории⁷. Сводная матрица оптимизаций всех трёх компиляторов — там же⁸. Ниже приведены ключевые наблюдения.

1. Свёртка констант. Все три компилятора вычисляют $1 + 2 = 3$ на этапе компиляции даже при `-O0`: в листинге нет инструкции сложения, сразу появляется `movl $3`. Для вещественных констант при `-O2` картина различается: GCC хранит результат $2.4 + 6.3 = 8.7$ в секции `.rodata` и загружает через `movsd .LC1(%rip)`, что требует обращения к памяти. Clang и ICX встраивают IEEE 754-представление (`0x4021666666666666`) прямо в инструкцию `movabsq` — обращения к памяти нет. В данном случае Clang и ICX эффективнее GCC.

2. Арифметические тождества. GCC при `-O0` уже убирает бессмысленные операции: вместо `i+0`, `i/1` и `i*1` генерирует простой `mov`. Clang и ICX при `-O0` следуют исходнику буквально: `addl $0, %eax` (сложение с нулём), `idivl $1` (деление — 20–90 тактов), `shll $0, %eax` (сдвиг на 0 бит), `imull $0` (умножение вместо `movl $0`). При `-O2` все три компилятора корректно оптимизируют эти конструкции. Поведение Clang и ICX при `-O0` удобнее для отладки, но очевидно неэффективно.

3. Снижение мощности. Замена `4 * j5` на `j5 << 2` выполняется всеми тремя компиляторами уже при `-O0`: умножение на степень двойки эквивалентно битовому сдвигу (1 такт против 3–5 тактов). GCC использует мнемонику `sall`, Clang и ICX — `shll`: это одна и та же инструкция `x86` (опкод `0xc1/4`).

4. Лишнее присваивание. При `-O0` все компиляторы генерируют два одинаковых `movl $1, k3`. При `-O2` остаётся одна инструкция: второй `mov` перезаписывает первый до его чтения — первый является мёртвым хранилищем (`dead store`) и убирается.

5. Мёртвый код. Блок `if(0){printf(...)}` убирается всеми компиляторами уже при `-O0`. При `-O2` функция `dead_code` сводится к одному `ret`. Примечательно: GCC добавляет перед `ret` инструкцию `endbr64` — метку допустимой цели перехода для Intel CET (Control-flow Enforcement Technology), защищающей от атак типа ROP. Clang и ICX эту инструкцию не генерируют — потенциально менее защищённый, но на одну инструкцию короткий код.

6. Ненужный цикл и хвостовые вызовы. Функция `unnecessary_loop` содержит цикл из 5 итераций, где `x = 0`, значит `k5 = j5` на каждой итерации. При `-O2` все три компилятора полностью убирают цикл и применяют оптимизацию хвостового вызова: `call printf; ret` заменяется на `jmp printf`, экономя кадр стека. GCC вызывает защищённую версию `__printf_chk`, Clang и ICX — стандартный `printf`.

7. Размотка цикла — ключевое различие компиляторов. GCC и Clang при `-O2` сохраняют цикл `loop_unrolling` из 6 итераций. ICX разворачивает его полностью — 6 прямых инструкций записи, при этом объединяя запись двух элементов `short` в одну инструкцию `movq` (8 байт). Устраняются 6 инструкций `cmpl` и 6 инструкций `jle`; число обращений к памяти сокращается вдвое. Аналогичная размотка применяется в `loop_jamming`: ICX раскрывает оба цикла по 5 итераций в прямую последовательность с предвычисленными константами.

8. Инвариант цикла и предвычисление массива. Выражение `j * k` в цикле по `ivector2` при `-O0` вычисляется на каждой итерации. При `-O2` все три компилятора выносят умножение за пределы цикла (`loop-invariant code motion`). Цикл `ivector4[i] = i * 2` при `-O2` полностью убирается: значения `{0, 2, 4, 6, 8, 10}` записываются двумя инструкциями. GCC хранит их в `.rodata`, Clang и ICX встраивают 64-битную константу `0x0006000400020000` прямо в `movabsq`.

9. Замена деления умножением (только ICX). Операция `m3 = (h3+k3)/i3` у GCC и Clang остаётся инструкцией `idivl`. ICX при `-O2` заменяет её:

```
; GCC/Clang:
idivl i3(%rip)          ; 20–90 тактов

; ICX -O2: деление на 3 без idiv
```

⁶Таблицы времени выполнения: https://github.com/Andrew2114/Compiler_optimization/blob/main/tables/04_execution_times.md.

⁷Таблицы ассемблерного анализа — GCC, Clang, ICX: https://github.com/Andrew2114/Compiler_optimization/blob/main/tables/04_1_asm_analys.md, https://github.com/Andrew2114/Compiler_optimization/blob/main/tables/04_2_asm_analys.md, https://github.com/Andrew2114/Compiler_optimization/blob/main/tables/04_3_asm_analys.md.

⁸Сводная таблица оптимизаций: https://github.com/Andrew2114/Compiler_optimization/blob/main/tables/05_comparison_table.md.

```
movl $2863311531, %esi ; = ceil(2^33 / 3)
imulq %rax, %rsi      ; битное64- умножение
shrq $33, %rsi        ; арифметический сдвиг
```

Математически: $\lfloor n/3 \rfloor = \lfloor n \cdot \lceil 2^{33}/3 \rceil / 2^{33} \rfloor$. `imulq` + `shrq` выполняются за 4–6 тактов — ускорение в 5–15 раз. GCC и Clang такую замену не производят.

10. Инициализация FPU (только ICX). ICX при `-O2` добавляет в начало `main`:

```
stmxcsr 4(%rsp) ; сохранить регистр управления SSE
orl $32832, 4(%rsp) ; установить биты DAZ и FTZ (0x8040)
ldmxcsr 4(%rsp) ; загрузить обратно
```

Биты DAZ (Denormals Are Zero) и FTZ (Flush To Zero) переводят блок SSE в режим, где денормализованные числа заменяются нулём. Обработка денормалей через микрокод занимает до 100 тактов; в режиме DAZ/FTZ этого не происходит. GCC и Clang данную инициализацию не выполняют.

11. Адресация переменных. ICX при `-O0` использует абсолютную адресацию (`movl k5, %eax` — 8 байт на адрес), тогда как GCC и Clang — RIP-относительную (`movl k5(%rip), %eax` — 4 байта на смещение). При `-O2` ICX переходит на RIP-относительную адресацию, что стандартно для позиционно-независимого кода (PIC).

Вывод по анализу листингов. Наибольшую агрессивность при `-O2` демонстрирует ICX: единственный полностью разматывает короткие циклы, заменяет деление умножением и инициализирует FPU. GCC выделяется частичной оптимизацией уже при `-O0` и добавлением защиты CET (`endbr64`). Clang при `-O0` строго следует исходнику, при `-O2` — эффективен наравне с GCC, но уступает ICX в агрессивности преобразования циклов. Все три компилятора одинаково хорошо справляются с устранением мёртвого кода, ненужных циклов и лишних присваиваний.

4. Заключение

В ходе выполнения работы получены следующие результаты:

- Тестовая программа `optbench.c` скомпилирована тремя компиляторами (GCC 13.3.0, Clang 18.1.3, Intel ICX 2026.0.0) с флагами `-O0`, `-O2` и `-O3`; для каждого варианта зафиксированы размер исполняемого файла и время выполнения.
- Получены и проанализированы ассемблерные листинги для 14 типов оптимизаций; составлены три сравнительные таблицы в формате «С-код — `-O0` — `-O2`» с подробными пояснениями.
- Установлено, что GCC при `-O0` уже частично оптимизирует код (убирает тождества `+0`, `/1`, `*1`), тогда как Clang и ICX строго следуют исходнику, генерируя бесполезные инструкции `addl $0, idivl $1, shll $0, imull $0`.
- Выявлены уникальные оптимизации ICX: полная размотка коротких циклов (`loop_unrolling`, `loop_jamming`), замена деления умножением на магическое число, объединение записей через `movq` и инициализация FPU-режима DAZ/FTZ.
- Показано, что при `-O2` все три компилятора одинаково эффективно устраняют мёртвый код и мёртвые хранилища, ликвидируют ненужные циклы с применением хвостового вызова, выносят инварианты и предвычисляют константные массивы.
- Зафиксировано, что флаги оптимизации слабо влияют на время выполнения данного бенчмарка: программа `IO-bound` (многократные вызовы `printf`), системные вызовы составляют до 90 % времени.

Репозиторий с исходным кодом, ассемблерными листингами и всеми таблицами: https://github.com/Andrew2114/Compiler_optimization.

Список литературы

- [1] Ахо А., Лам М., Сети Р., Ульман Дж. *Компиляторы: принципы, технологии, инструменты*. М.: Вильямс, 2008. Гл. 8–9.
- [2] Брайант Р., О’Халларон Д. *Компьютерные системы: архитектура и программирование*. М.: Вильямс, 2016. Гл. 3, 5.
- [3] GCC Team. *Options That Control Optimization* // GCC 14 Documentation. URL: <https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html> (дата обращения: 17.04.2026).
- [4] LLVM Team. *Clang Command Guide* // Clang Documentation. URL: <https://clang.llvm.org/docs/CommandGuide/clang.html> (дата обращения: 09.03.2026).
- [5] Intel Corporation. *Intel oneAPI DPC++/C++ Compiler Developer Guide*. URL: <https://www.intel.com/content/www/us/en/docs/dpcpp-cpp-compiler/developer-guide-reference/> (дата обращения: 22.05.2026).
- [6] Fog A. *Optimizing subroutines in assembly language*. Technical University of Denmark, 2024. URL: https://www.agner.org/optimize/optimizing_assembly.pdf (дата обращения: 09.03.2026).